

Regular: Context-Aware Page Ranking for DRAM Demotion in Hybrid Memory Systems

Anonymous Author(s)

ABSTRACT

DRAM–NVM hybrid memory systems address growing capacity demands by combining fast but capacity-limited DRAM with high-density non-volatile memory (NVM). To use limited DRAM capacity effectively and reduce costly NVM writes, an appropriate DRAM-resident page must be selected for demotion to NVM, while hot and write-intensive pages are retained in DRAM. Existing heuristics use coarse access signals, while learning-based policies suffer from limited context, objective mismatch, or high online overhead. We propose **CAPD**, a **C**ontext-**A**ware **P**age-**D**emotion policy that ranks DRAM-resident candidate pages. CAPD encodes recent access history with a single-layer Transformer and lets each candidate page extract candidate-specific context for demotion scoring. CAPD is trained offline with reference rankings derived from trace replay and is invoked only at demotion events. On PARSEC-derived traces, CAPD reduces weighted replay cost under DRAM capacity pressure, including a 10.83% reduction over the best classical baseline on the streamcluster pressure window.

1 INTRODUCTION

With the growth of data-intensive applications such as machine learning [5, 22], graph analytics [6, 34], and in-memory analytics [41, 42], modern systems increasingly demand larger main-memory capacity and higher memory performance. However, DRAM scaling has become increasingly constrained [12, 26], while non-volatile memory (NVM) provides higher density and lower cost per bit [12, 19, 28] but suffers from higher write latency [12, 39] and limited write endurance [13, 44]. Therefore, DRAM–NVM hybrid memory systems [3, 24, 25, 29, 38], which combine low-latency DRAM with high-density NVM, have emerged as a promising main-memory design.

In such systems, DRAM typically serves as a fast tier for hot and write-intensive pages [1, 14, 29, 40], while NVM provides a large-capacity tier for colder pages [15, 16, 21, 30]. Since DRAM is a capacity-limited fast tier [11, 20, 29] and NVM writes are costly [17, 32], an effective demotion policy should move colder and less write-intensive pages to NVM, while keeping hot or write-intensive pages in DRAM. Choosing a poor demotion victim—for example, a page that is reused soon or written frequently—can increase slow-tier accesses, NVM write traffic, and device wear [14, 17, 29, 44].

Traditional replacement policies based on fixed heuristic rules, such as LRU, LFU, and CLOCK [2, 10], often struggle with dynamic access behavior [8, 14]. Learning-based cache-replacement, prefetching, and hybrid-memory policies [7, 33, 37] can exploit access history or program context, but their objectives generally target cache utility, future access prediction, or tiering rather than demotion-time ranking of the current DRAM candidates [4, 9, 18, 43].

These limitations leave a gap for a demotion-time policy that uses richer runtime context while keeping the decision scope bounded to the pages currently eligible for demotion. Rather than predicting global page hotness, such a policy should compare the current DRAM candidates under the same access context and select the page whose demotion is expected to cause fewer future slow-tier accesses and lower NVM write cost.

To fill this gap, we present CAPD, a **C**ontext-**A**ware **P**age **D**emotion policy for DRAM–NVM hybrid memory systems. CAPD embeds address identifiers, program counters (PCs), and read/write flags into multi-dimensional access representations, and uses a single-layer Transformer [36] to encode the recent access sequence. For each DRAM candidate, CAPD uses its page-level representation to extract candidate-specific context from the encoded sequence, and scores this context to select the page with the highest demotion-priority score.

In summary, this paper makes the following contributions:

- We formulate DRAM-to-NVM page demotion in DRAM–NVM hybrid memory systems as a context-aware candidate-page ranking problem. Instead of relying on fixed recency- or frequency-based victim-selection rules, CAPD directly ranks the current DRAM-resident candidates under capacity pressure using recent access context and page-level states.
- We design a candidate-specific demotion scoring mechanism that encodes recent access records and uses candidate-to-sequence attention to produce a demotion-priority score for each DRAM candidate.
- We develop a replay-derived demotion objective and evaluate CAPD with PARSEC-derived trace replay. The objective incorporates future inactivity, coldness, and write intensity. The results show that CAPD reduces weighted replay cost under sustained DRAM pressure and highlights workload dependence.

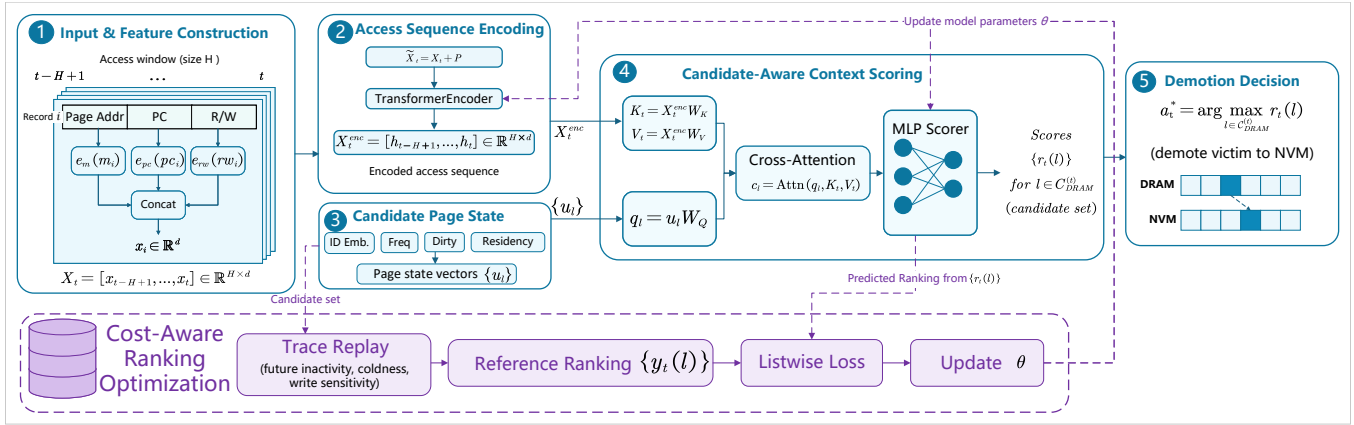


Figure 1: Overall framework of CAPD.

2 BACKGROUND AND RELATED WORK

DRAM-to-NVM demotion differs from conventional cache or page replacement because a demotion victim should be selected based not only on reuse behavior but also on write intensity and demotion-related cost [14, 17, 29, 32]. Representative policies such as LRU, LFU, CLOCK, and ARC are lightweight, but still rely largely on coarse recency- or frequency-based signals [2, 10, 23]. Learning-based approaches can improve adaptability by exploiting access history [18, 31, 33, 37] or program context [7], but many optimize cache hit rate, caching utility, or future access prediction [7, 9, 18, 33, 35]. Models that exploit long access histories may also incur nontrivial online overhead due to large history windows or complex inference [27, 33]. Hybrid-memory learners such as Kleio and PatternS [4, 43] predict page hotness or page-group behavior for tiering, but do not directly rank DRAM candidates at demotion time.

These limitations motivate CAPD’s candidate-ranking formulation. At each demotion event, the memory manager only needs to select one victim from a bounded set of eligible DRAM-resident pages, making demotion a relative ranking problem rather than an absolute hotness-prediction problem. CAPD therefore focuses on ranking the current demotion candidates using recent access context and page-level states.

3 DESIGN

3.1 Overall Framework

Figure 1 illustrates the overall framework of CAPD. CAPD treats DRAM page demotion as a context-aware candidate ranking problem. It consists of four components: Multi-dimensional Feature Construction, Access Sequence Encoding, Candidate Page Ranking and Demotion Decision, and Replay-Derived Ranking Optimization. It encodes recent accesses with a lightweight Transformer, lets each candidate

page query the encoded sequence to obtain a candidate-specific context, and scores the context for demotion. Training uses offline trace replay to construct reference rankings for optimization.

3.2 Problem Definition

We consider a DRAM–NVM hybrid memory system, where DRAM offers low-latency access but has limited capacity, while NVM provides larger capacity but higher write overhead. When DRAM capacity pressure triggers a demotion decision at time t , the system must select one DRAM-resident candidate page as the victim and demote it to NVM.

At each demotion point t , CAPD observes the recent access history $\mathcal{H}_t$ and a candidate set $C_{\text{DRAM}}^{(t)}$ of DRAM-resident pages eligible for demotion. In our implementation, $C_{\text{DRAM}}^{(t)}$ is bounded by selecting pages from the LRU tail of the DRAM-resident set. We use s_t to denote the demotion state induced by the recent access history and the candidate set. CAPD assigns each candidate page l a demotion-priority score $r_t(l)$ and selects the highest-scoring page as the victim:

$$a_t^* = \arg \max_{l \in C_{\text{DRAM}}^{(t)}} r_t(l), \quad (1)$$

where $r_t(l)$ denotes the priority of demoting page l from DRAM to NVM.

3.3 Multi-dimensional Feature Construction

CAPD constructs multi-dimensional access features from memory-access records. Each record contains a page identifier m_t derived from the virtual address, a program counter identifier pc_t , and a read/write flag rw_t , where $rw_t = 0$ for reads and $rw_t = 1$ for writes. These fields capture page-level locality, program context, and access type, respectively.

CAPD embeds and concatenates these fields as:

$$x_t = e_m(m_t) \parallel e_{pc}(pc_t) \parallel e_{rw}(rw_t) \in \mathbb{R}^d, \quad (2)$$

where $\parallel$ denotes concatenation and $d = d_m + d_{pc} + d_{rw}$. At a demotion point t , CAPD constructs a historical access window from the most recent H access representations:

$$X_t = [x_{t-H+1}, \dots, x_t]^\top \in \mathbb{R}^{H \times d}, \quad (3)$$

where H denotes the sliding-window length.

3.4 Access Sequence Encoding

At each demotion point t , CAPD treats the historical window X_t as an ordered sequence and adds positional encodings:

$$\tilde{X}_t = X_t + P, \quad (4)$$

where $P \in \mathbb{R}^{H \times d}$ denotes the positional encoding.

A lightweight single-layer Transformer encoder then produces contextualized access representations:

$$X_t^{enc} = \text{TEnc}_1(\tilde{X}_t) = [h_{t-H+1}, h_{t-H+2}, \dots, h_t]^\top \in \mathbb{R}^{H \times d}. \quad (5)$$

Here, each h_i denotes a contextualized representation.

3.5 Candidate Page Ranking and Demotion Decision

When a DRAM page demotion decision is triggered, CAPD constructs a page-level representation u_l for each candidate page $l \in C_{\text{DRAM}}^{(t)}$ from its page identifier embedding and lightweight state features, including recent access frequency, dirty status, normalized residency duration, and normalized position within the LRU-tail candidate set. Rather than compressing the access history into a single shared context for all candidates, CAPD uses each candidate page representation as a query to extract a candidate-specific context from the encoded access sequence. CAPD projects the candidate representation into a query and the encoded access sequence into keys and values:

$$q_l = u_l W_Q, \quad K_t = X_t^{enc} W_K, \quad V_t = X_t^{enc} W_V. \quad (6)$$

CAPD then computes the candidate-specific context of page l through cross-attention:

$$c_l = \text{softmax} \left(\frac{q_l K_t^\top}{\sqrt{d_a}} \right) V_t, \quad (7)$$

where d_a denotes the attention dimension. The resulting vector c_l is the candidate-specific context for page l .

CAPD scores this context with an MLP:

$$r_t(l) = \text{MLP}(c_l), \quad (8)$$

where $r_t(l)$ denotes the DRAM-to-NVM demotion-priority score. Because c_l is extracted using the query derived from u_l , the score is conditioned on both the encoded access sequence and the candidate page state.

Ultimately, CAPD demotes the highest-scoring page:

$$a_t^* = \arg \max_{l \in C_{\text{DRAM}}^{(t)}} r_t(l). \quad (9)$$

3.6 Replay-Derived Ranking Optimization

CAPD is trained offline using trace replay. At each demotion point t , replay provides a candidate set $C_{\text{DRAM}}^{(t)}$ and assigns each candidate page l a reference demotion relevance score:

$$y_t(l) = \lambda_1 \hat{d}_t(l) + \lambda_2 \hat{q}_t(l) - \lambda_3 \hat{w}_t(l), \quad (10)$$

where $\hat{d}_t(l)$, $\hat{q}_t(l)$, and $\hat{w}_t(l)$ denote normalized future inactivity, coldness, and write intensity within a lookahead window of length $L = 256$. Intuitively, a page receives a higher relevance score if it has longer future inactivity, lower future access frequency, and lower future write intensity. We use $(\lambda_1, \lambda_2, \lambda_3) = (1, 1, 4)$ and normalize the resulting scores within each candidate set.

Given predicted demotion scores $r_t(l)$, CAPD normalizes them over the same candidate set:

$$\pi_\theta(a_t = l \mid s_t) = \frac{\exp(r_t(l))}{\sum_{j \in C_{\text{DRAM}}^{(t)}} \exp(r_t(j))}. \quad (11)$$

CAPD then optimizes a listwise approximate-NDCG loss:

$$\mathcal{L}_{\text{rank}}(s_t, \pi_\theta) = -\frac{\text{DCG}(s_t, \pi_\theta)}{\text{IDCG}(s_t) + \epsilon}. \quad (12)$$

This objective encourages pages with higher replay-derived relevance scores to be ranked closer to the top. Appendix A gives the detailed definitions of the replay labels and differentiable ranking loss.

3.7 Complexity and System Overhead

For each demotion decision, CAPD encodes an access window of length H , computes candidate-specific contexts through cross-attention, and scores all pages in $C_{\text{DRAM}}^{(t)}$. With feature dimension d and attention dimension d_a , the per-decision inference cost is

$$O(H^2 d + H d^2 + |C_{\text{DRAM}}^{(t)}| H d + |C_{\text{DRAM}}^{(t)}| d^2), \quad (13)$$

where the first two terms come from the single-layer Transformer encoder, the third term comes from candidate-specific cross-attention over the access window, and the last term comes from lightweight page scoring. Positional encoding adds only $O(Hd)$ lower-order cost.

4 EVALUATION

4.1 Experimental Setup

We evaluate CAPD using trace replay on PARSEC-derived memory traces. Traces are collected with DynamoRIO/drmemtrace and converted into (PC, Address, RW) records. We align each recorded virtual

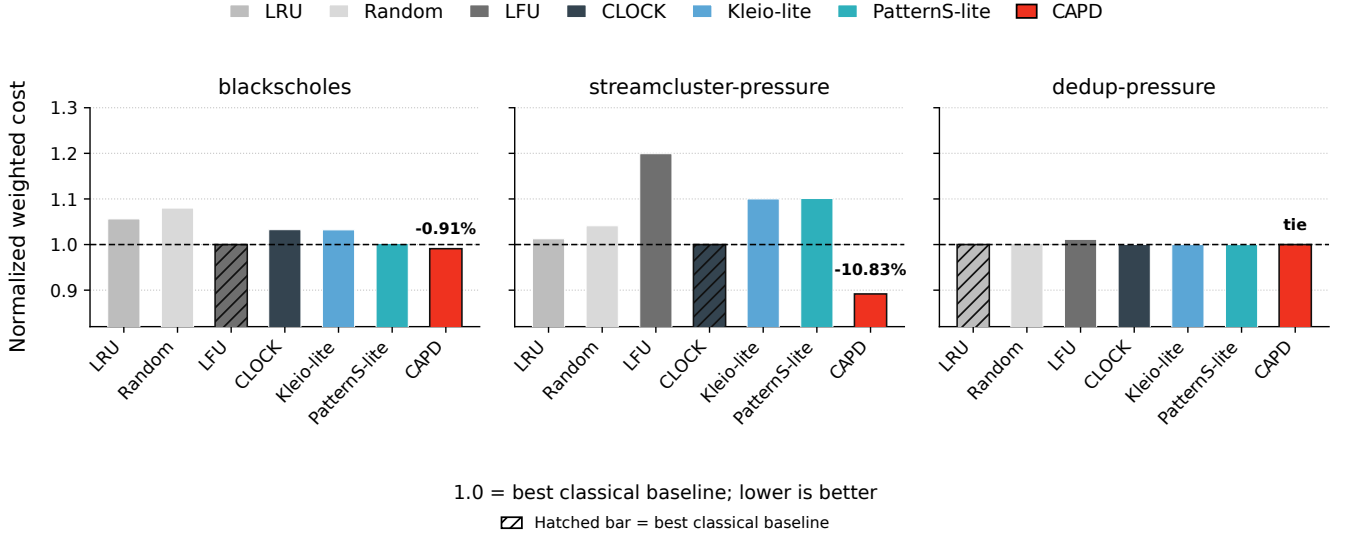


Figure 2: Overall weighted-cost comparison under the pre-specified default seed 3136859. Costs are normalized to the best classical baseline in each workload; lower is better. The hatched bar marks the best classical baseline.

address to a 4KB page and use the resulting virtual page identifier in replay; the prototype does not reconstruct physical addresses. For standard experiments, each workload contains 1M memory references and is split chronologically into 800K training, 100K validation, and 100K test references. For workloads with little demotion activity in the standard test split, we also report fixed pressure windows selected before CAPD training or evaluation. We select such windows using only LRU-triggered demotion counts under a 16-page DRAM tier, and replay the same selected windows for all policies. Appendix B gives the detailed window-selection procedure and selected ranges. Unless otherwise stated, the DRAM tier holds 16 pages, the history window contains $H = 10$ accesses, the demotion candidate set contains 8 pages from the LRU tail, and the replay lookahead is $L = 256$. We compare CAPD against LRU, Random, LFU, CLOCK, and two replay-compatible learned baselines: Kleio-lite [4] and PatternS-lite [43]. All main results use the pre-specified default seed 3136859. We use the following weighted replay cost:

$$C = N_{\text{hit}} + 2N_{\text{nvm_read}} + 8N_{\text{nvm_write}} + 10N_{\text{demotion}}. \quad (14)$$

This metric assigns higher cost to NVM writes and DRAM-to-NVM demotions than to DRAM hits, reflecting the asymmetric cost structure of DRAM–NVM hybrid memory. Lower cost is better.

4.2 Main Results

Figure 2 summarizes the main replay results. CAPD improves the weighted cost on workloads where replacement decisions

expose a real tradeoff between hit rate, NVM writes, and demotions. On streamcluster-pressure, CAPD reduces normalized cost by 10.83% over the best classical baseline and by 18.93% over the best learned baseline. On blackscholes, the default-seed run slightly improves over the best baseline. We therefore treat blackscholes as a near-tie rather than as primary evidence for CAPD’s benefit.

Table 1 gives the same comparison in absolute weighted cost. The strongest result is on streamcluster-pressure, where CAPD reduces demotions from 8937 under CLOCK to 5981 while also increasing the hit rate from 95.52% to 97.00%. This shows that CAPD is not merely reducing writes by sacrificing locality; it learns demotion choices that jointly reduce NVM reads, NVM writes, and demotions. On blackscholes, CAPD is close to LFU in hit rate but incurs substantially fewer NVM writes, leading to a small overall cost reduction. On dedup-pressure, almost all competitive policies converge to the same behavior, leaving little room for improvement.

4.3 Sensitivity and Robustness

Table 2 summarizes capacity sensitivity on streamcluster-pressure, where Δ denotes the relative weighted-cost change of CAPD over the best rule-based policy. CAPD remains beneficial with 8 and 16 DRAM pages, reducing weighted cost by about 10% relative to the best rule-based policy. When the DRAM capacity increases to 32 pages, the miss stream and demotion frequency become much smaller, and CAPD converges to near parity with the best classical baseline. This trend

Table 1: Main replay metrics under the pre-specified default seed 3136859. Lower values are better for weighted cost, NVM reads, NVM writes, and demotions; higher values are better for hit rate.

Workload	Policy	Weighted cost	Hit rate (%)	NVM reads	NVM writes	Demotions
blackscholes	LRU	112,958	98.89	970	144	1,098
	Random	115,505	98.66	1,200	145	1,329
	LFU	106,952	99.47	305	221	510
	CLOCK	110,437	99.09	798	107	889
	Kleio-lite	110,390	99.04	956	2	942
	PatternS-lite	107,207	99.36	594	49	627
	CAPD	105,983	99.46	509	32	525
streamcluster-pressure	LRU	305,562	95.35	8,707	585	9,276
	Random	314,183	95.01	9,194	777	9,955
	LFU	361,877	92.84	13,587	740	14,311
	CLOCK	301,767	95.52	8,379	574	8,937
	Kleio-lite	331,941	94.19	10,932	697	11,613
	PatternS-lite	332,226	94.15	11,063	629	11,676
	CAPD	269,095	97.00	5,449	548	5,981
dedup-pressure	LRU	201,567	99.95	4	99	87
	Random	202,076	99.93	41	105	130
	LFU	203,845	99.88	4	233	221
	CLOCK	201,584	99.95	4	100	88
	Kleio-lite	201,567	99.95	4	99	87
	PatternS-lite	201,567	99.95	4	99	87
	CAPD	201,567	99.95	4	99	87

Table 2: Capacity sensitivity on streamcluster-pressure.

DRAM pages	Best rule	CAPD	Δ	CAPD demotions
8	369999	332660	-10.09%	11738
16	301767	269095	-10.83%	5981
32	216577	217348	+0.36%	1566

is expected: learned victim ranking is most useful when DRAM pressure is high enough that a poor demotion decision can cause repeated NVM accesses or additional demotions. Appendix C reports additional seed-stability and cost-weight robustness results. Across three training seeds, CAPD consistently improves over the best classical baseline on streamcluster-pressure, while blackscholes remains near parity and canneal is a stable negative case. Under alternative cost settings, CAPD remains beneficial on streamcluster-pressure, suggesting that the main positive result is not tied to a single hand-picked cost weighting.

4.4 Discussion

The results show that CAPD is most useful when demotion choices materially affect future slow-tier accesses and demotion cost. On streamcluster-pressure, CAPD improves hit rate while reducing NVM reads, NVM writes, and demotions, and this trend remains stable across seeds and cost settings. Its benefit is not universal: blackscholes and dedup-pressure are near-tie cases, while canneal regresses by 16.81% on average across three seeds. CAPD therefore supports context-aware candidate ranking as a promising direction, but not as a replacement for all classical policies. We

interpret these results as framework-level evidence rather than a component-wise attribution study. CAPD’s current evaluation shows that the complete context-aware ranking pipeline can improve replay behavior under sustained DRAM pressure. Integrating CAPD into a runtime memory manager and measuring end-to-end overhead remain future work.

5 CONCLUSION

This paper presented CAPD, a context-aware page-demotion policy for DRAM–NVM hybrid memory systems. CAPD formulates DRAM victim selection as candidate-page ranking, using recent access history and page-level states to derive candidate-specific demotion context. Offline replay provides reference rankings, while inference is invoked only at demotion events. On PARSEC-derived traces, CAPD reduces weighted replay cost by 10.83% on streamcluster-pressure, remains near parity on blackscholes and dedup-pressure, and underperforms on canneal. These results suggest that context-aware candidate ranking is promising under sustained DRAM pressure, especially when demotion decisions affect both future slow-tier accesses and page-movement cost. At the same time, CAPD’s workload-dependent behavior shows that learned demotion policies should be evaluated not only by their best-case improvements, but also by the trace regimes in which classical policies remain competitive.

REFERENCES

- [1] Neha Agarwal and Thomas F Wenisch. 2017. Thermostat: Application-transparent page management for two-tiered main memory. In *Proceedings of the Twenty-Second International Conference on Architectural*

- Support for Programming Languages and Operating Systems*. 631–644.
- [2] Laszlo A. Belady. 1966. A study of replacement algorithms for a virtual-storage computer. *IBM Systems journal* 5, 2 (1966), 78–101.
 - [3] Gaurav Dhiman, Raid Ayoub, and Tajana Rosing. 2009. PDRAM: A hybrid PRAM and DRAM main memory system. In *Proceedings of the 46th Annual Design Automation Conference*. 664–669.
 - [4] Thaleia Dimitra Doudali, Sergey Blagodurov, Abhinav Vishnu, Sudhanva Gurumurthi, and Ada Gavrilovska. 2019. Kleio: A Hybrid Memory Page Scheduler with Machine Intelligence. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*. 37–48. <https://doi.org/10.1145/3307681.3325398>
 - [5] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. 2024. AI and memory wall. *IEEE Micro* 44, 3 (2024), 33–39.
 - [6] Joseph E. Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. PowerGraph: Distributed Graph-Parallel Computation on Natural Graphs. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI '12)*. 17–30.
 - [7] Milad Hashemi, Kevin Swersky, Jamie Smith, Grant Ayers, Heiner Litz, Jichuan Chang, Christos Kozyrakis, and Parthasarathy Ranganathan. 2018. Learning memory access patterns. In *International Conference on Machine Learning*. PMLR, 1919–1928.
 - [8] Taekyung Heo, Yang Wang, Wei Cui, Jaehyuk Huh, and Lintao Zhang. 2022. Adaptive Page Migration Policy with Huge Pages in Tiered Memory Systems. *IEEE Trans. Comput.* 71, 1 (2022), 53–68. <https://doi.org/10.1109/TC.2020.3036686>
 - [9] Xinyue Hu, Eman Ramadan, Wei Ye, Feng Tian, and Zhi-Li Zhang. 2022. Raven: belady-guided, predictive (deep) learning for in-memory and content caching. In *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies*. 72–90.
 - [10] Song Jiang, Feng Chen, and Xiaodong Zhang. 2005. CLOCK-Pro: An Effective Improvement of the CLOCK Replacement. In *USENIX ATC, General Track*. 323–336.
 - [11] Sudarsun Kannan, Ada Gavrilovska, Vishal Gupta, and Karsten Schwan. 2017. Heteroos: Os design for heterogeneous memory management in datacenter. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 521–534.
 - [12] Benjamin C. Lee, Engin Ipek, Onur Mutlu, and Doug Burger. 2009. Architecting Phase Change Memory as a Scalable DRAM Alternative. In *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*.
 - [13] Benjamin C Lee, Ping Zhou, Jun Yang, Youtao Zhang, Bo Zhao, Engin Ipek, Onur Mutlu, and Doug Burger. 2010. Phase-change technology and the future of main memory. *IEEE micro* 30, 1 (2010), 143.
 - [14] Soyoon Lee, Hyokyung Bahn, and Sam H Noh. 2014. CLOCK-DWF: A write-history-aware page replacement algorithm for hybrid PCM and DRAM memory architectures. *IEEE Trans. Comput.* 63, 9 (2014), 2187–2200.
 - [15] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. 2023. Memtis: Efficient memory tiering with dynamic page classification and page size determination. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 17–34.
 - [16] Huaicheng Li, Daniel S. Berger, Stanko Novakovic, Lisa Hsu, Dan Ernst, Pantea Zardoshti, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*. 574–587. <https://doi.org/10.1145/3575693.3578835>
 - [17] Yang Li, Saugata Ghose, Jongmoo Choi, Jin Sun, Hui Wang, and Onur Mutlu. 2017. Utility-based hybrid memory management. In *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 152–165.
 - [18] Evan Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. 2020. An imitation learning approach for cache replacement. In *International Conference on Machine Learning*. PMLR, 6237–6247.
 - [19] Hai-Kun Liu, Di Chen, Hai Jin, Xiao-Fei Liao, Bingsheng He, Kan Hu, and Yu Zhang. 2021. A survey of non-volatile main memory technologies: State-of-the-arts, practices, and future directions. *Journal of Computer Science and Technology* 36, 1 (2021), 4–32.
 - [20] Lei Liu, Shengjie Yang, Lu Peng, and Xinyu Li. 2019. Hierarchical hybrid memory management in OS for tiered memory systems. *IEEE Transactions on Parallel and Distributed Systems* 30, 10 (2019), 2223–2236.
 - [21] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2023. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 742–755.
 - [22] Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, et al. 2020. MLPerf Training Benchmark. In *Proceedings of Machine Learning and Systems*, Vol. 2. 336–349.
 - [23] Nimrod Megiddo and Dharmendra S Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *2nd USENIX Conference on File and Storage Technologies (FAST 03)*.
 - [24] Mitesh R Meswani, Sergey Blagodurov, David Roberts, John Slice, Mike Ignatowski, and Gabriel H Loh. 2015. Heterogeneous memory architectures: A HW/SW approach for mixing die-stacked and off-package memories. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 126–136.
 - [25] Jeffrey C. Mogul, Eduardo Argollo, Mehul M. Shah, and Paolo Fabbroschi. 2009. Operating System Support for NVM+DRAM Hybrid Main Memory. In *Proceedings of the 12th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*.
 - [26] Onur Mutlu. 2013. Memory scaling: A systems architecture perspective. In *2013 5th IEEE International Memory Workshop*. 21–25. <https://doi.org/10.1109/IMW.2013.6582088>
 - [27] Songwen Pei, Jianan Li, Yihuan Qian, Jie Tang, and Jean-Luc Gaudiot. 2022. TransMigrator: A Transformer-Based Predictive Page Migration Mechanism for Heterogeneous Memory. In *Network and Parallel Computing: 19th IFIP WG 10.3 International Conference, NPC 2022, Jinan, China, September 24–25, 2022, Proceedings* (Jinan, China). Springer-Verlag, Berlin, Heidelberg, 180–191. https://doi.org/10.1007/978-3-031-21395-3_17
 - [28] Moinuddin K. Qureshi, Vijayalakshmi Srinivasan, and Jude A. Rivers. 2009. Scalable High Performance Main Memory System Using Phase-Change Memory Technology. In *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*.
 - [29] Luiz E Ramos, Eugene Gorbato, and Ricardo Bianchini. 2011. Page placement in hybrid memory systems. In *Proceedings of the international conference on Supercomputing*. 85–95.
 - [30] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. 2021. Hemem: Scalable tiered memory management for big data applications and real nvm. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 392–407.
 - [31] Liana V Rodriguez, Farzana Beente Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. 2021. Learning Cache Replacement with CACHEUS. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 341–354.

- [32] Reza Salkhordeh and Hossein Asadi. 2016. An operating system level data migration scheme in hybrid DRAM-NVM memory architecture. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 936–941.
- [33] Zhan Shi, Xiangru Huang, Akanksha Jain, and Calvin Lin. 2019. Applying deep learning to the cache replacement problem. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 413–425.
- [34] Julian Shun and Guy E. Blelloch. 2013. Ligra: A Lightweight Graph Processing Framework for Shared Memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '13)*. ACM, 135–146. <https://doi.org/10.1145/2442516.2442530>
- [35] Zhenyu Song, Kevin Chen, Nikhil Sarda, Deniz Altınbüken, Eugene Brevdo, Jimmy Coleman, Xiao Ju, Pawel Jurczyk, Richard Schooler, and Ramki Gummadi. 2023. HALP: Heuristic Aided Learned Preference Eviction Policy for YouTube Content Delivery Network. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 1149–1163.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. 5998–6008.
- [37] Giuseppe Vietri, Liana V Rodriguez, Wendy A Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving Cache Replacement with ML-Based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*.
- [38] Kai Wu, Yingchao Huang, and Dong Li. 2017. Unimem: Runtime Data Management on Non-Volatile Memory-Based Heterogeneous Main Memory. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, Article 58, 14 pages. <https://doi.org/10.1145/3126908.3126923>
- [39] Jian Yang, Juno Kim, Morteza Hoseinzadeh, Joseph Izraelevitz, and Steve Swanson. 2020. An empirical guide to the behavior and use of scalable persistent memory. In *18th USENIX Conference on File and Storage Technologies (FAST 20)*. 169–182.
- [40] HanBin Yoon, Justin Meza, Rachata Ausavarungnirun, Rachael A. Harding, and Onur Mutlu. 2012. Row Buffer Locality Aware Caching Policies for Hybrid Memories. In *2012 IEEE 30th International Conference on Computer Design (ICCD '12)*. 337–344. <https://doi.org/10.1109/ICCD.2012.6378661>
- [41] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *9th USENIX symposium on networked systems design and implementation (NSDI 12)*. 15–28.
- [42] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph E. Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A Unified Engine for Big Data Processing. *Commun. ACM* 59, 11 (2016), 56–65. <https://doi.org/10.1145/2934664>
- [43] Yanjie Zhen, Weining Chen, Wei Gao, Ju Ren, Kang Chen, and Yu Chen. 2024. PatternS: An Intelligent Hybrid Memory Scheduler Driven by Page Pattern Recognition. *Journal of Systems Architecture* 153 (2024), 103178. <https://doi.org/10.1016/j.sysarc.2024.103178>
- [44] Ping Zhou, Bo Zhao, Jun Yang, and Youtao Zhang. 2009. A durable and energy efficient main memory using phase change memory technology. *ACM SIGARCH computer architecture news* 37, 3 (2009), 14–23.

A REPLAY LABEL AND RANKING-LOSS DETAILS

At each demotion point t , CAPD uses the future lookahead window of length $L = 256$ to construct replay-derived reference labels. For a candidate page l , let $\delta_t(l)$ be the distance from t to the next access to page l within the lookahead window. If page l is not accessed again in the window, we set $\hat{d}_t(l) = 1$; otherwise,

$$\hat{d}_t(l) = \frac{\delta_t(l)}{L}.$$

Thus, $\hat{d}_t(l)$ measures future inactivity: a larger value indicates that the page is reused later and is therefore a more suitable demotion candidate.

Coldness captures the total future access frequency rather than only the first future reuse. Let $f_t(l)$ be the number of future accesses to page l in the lookahead window. We define

$$\hat{q}_t(l) = 1 - \min\left(\frac{f_t(l)}{L}, 1\right).$$

A larger $\hat{q}_t(l)$ indicates that the page is accessed less frequently in the future window.

Write intensity measures future write pressure. Let $f_t^W(l)$ be the number of future writes to page l in the same lookahead window. We define

$$\hat{w}_t(l) = \min\left(\frac{f_t^W(l)}{L}, 1\right).$$

A larger $\hat{w}_t(l)$ lowers the demotion relevance because demoting a write-active page to NVM is undesirable.

The replay-derived demotion relevance score is

$$y_t(l) = \lambda_1 \hat{d}_t(l) + \lambda_2 \hat{q}_t(l) - \lambda_3 \hat{w}_t(l),$$

where we use $(\lambda_1, \lambda_2, \lambda_3) = (1, 1, 4)$ by default. Since the raw score may have different ranges across candidate sets, we apply min-max normalization within the valid candidate set:

$$\tilde{y}_t(l) = \frac{y_t(l) - \min_{j \in C_{\text{DRAM}}^{(t)}} y_t(j)}{\max_{j \in C_{\text{DRAM}}^{(t)}} y_t(j) - \min_{j \in C_{\text{DRAM}}^{(t)}} y_t(j) + \epsilon}.$$

A larger $\tilde{y}_t(l)$ indicates a better demotion candidate.

Given the predicted demotion scores $r_t(l)$, CAPD uses a listwise approximate-NDCG objective. For candidate l_i , the differentiable 1-based rank position is approximated as

$$\rho_t(l_i) = 1 + \sum_{j \neq i} \sigma(\alpha(r_t(l_j) - r_t(l_i))),$$

where $\sigma(\cdot)$ is the sigmoid function and $\alpha = 10$. The DCG term is computed as

$$\text{DCG}(s_t, \pi_\theta) = \sum_{l \in C_{\text{DRAM}}^{(t)}} \frac{\exp(\tilde{y}_t(l)) - 1}{\log(1 + \rho_t(l))}.$$

The ideal DCG, denoted $\text{IDCG}(s_t)$, is obtained by sorting candidates according to the replay-derived relevance $\tilde{y}_t(l)$. CAPD minimizes

$$\mathcal{L}_{\text{rank}}(s_t, \pi_\theta) = -\frac{\text{DCG}(s_t, \pi_\theta)}{\text{IDCG}(s_t) + \epsilon}.$$

This objective encourages candidates with higher replay-derived demotion relevance to appear closer to the top of the predicted ranking.

B EXPERIMENTAL SETUP DETAILS

B.1 Trace Processing and Data Splits

We collect user-space memory references with DynamoRIO/drmemtrace and convert the drmemtrace view output into $(PC, \text{Address}, RW)$ records. The recorded addresses are virtual addresses from the traced process. We align each address to a 4KB page using `page_shift=12` and use the resulting virtual page identifier in trace replay. The current prototype does not reconstruct physical addresses.

For the standard 1M-trace experiments, each workload contains 1,000,000 memory references and is split chronologically into 800K training, 100K validation, and 100K test references. All reported standard-split results use the held-out test window.

B.2 Pressure-Window Selection

For workloads with little demotion activity in the standard test split, we report pressure-window experiments. These windows are selected before training or evaluating CAPD and do not use CAPD performance. We scan chronological candidate windows with 100K and 200K references using a 10K-reference stride. For each candidate window, the last 10% is treated as the candidate test segment, and pressure is quantified as the number of LRU-triggered demotions under a 16-page DRAM tier. We select the window with the largest LRU-triggered demotion count and then use the same selected window for all policies.

The selected streamcluster pressure window uses references 0–270K for training, 270K–370K for validation, and 370K–570K for testing. The selected dedup pressure window uses references 0–700K for training, 700K–800K for validation, and 800K–1M for testing.

B.3 CAPD Configuration

Unless otherwise stated, the DRAM tier holds 16 pages, the history window contains $H = 10$ accesses, the demotion candidate set contains 8 pages, and the prediction lookahead is $L = 256$ accesses. At each DRAM miss with a full DRAM tier, the candidate set is constructed from the 8 pages at the LRU tail of the current DRAM-resident set.

CAPD uses 8-dimensional page-address embeddings, 8-dimensional PC embeddings, and 2-dimensional read/write embeddings, giving an 18-dimensional access representation. The access-sequence encoder is a one-layer Transformer with 2 attention heads and a 72-dimensional feed-forward layer. Each candidate page is represented by an 8-dimensional page embedding and four state features: recent access frequency, dirty status, normalized residency duration, and normalized LRU-tail rank. The candidate scorer uses 2-head cross-attention from candidate pages to the encoded access sequence, followed by an MLP with hidden dimensions 36 and 18. We train for 10 epochs with a batch size of 32.

B.4 Baselines and Random Seeds

We compare CAPD against four classical replacement policies: LRU, Random, LFU, and CLOCK. We also include two replay-compatible learned baselines. Kleio-lite follows Kleio’s neural page-hotness prediction and page-prioritization logic [4]. PatternS-lite follows PatternS’s page-pattern clustering and group-level hotness-prediction logic [43]. These baselines are replay-compatible adaptations rather than full-system reproductions.

Unless otherwise stated, the main figures and tables report the result of the pre-specified default training seed 3136859. This seed was fixed before inspecting the test results and is used consistently for the main replay comparison. Seed-stability results are reported separately.

C ADDITIONAL EXPERIMENTAL RESULTS

C.1 Capacity Sensitivity

We next vary the DRAM capacity on streamcluster-pressure, the workload with the clearest memory-pressure behavior. This experiment tests whether CAPD only wins at a single cache size or remains useful across pressure levels.

Table 2 shows that CAPD is most useful when DRAM pressure is high. With 8 and 16 DRAM pages, it reduces weighted cost by about 10% relative to the best rule-based policy. With 32 pages, the miss stream shrinks substantially and replacement decisions become less consequential; CAPD then converges to near parity with the best classical baseline. This behavior is expected for a learned demotion policy: its advantage appears when the cache is small enough that choosing the wrong victim causes repeated NVM accesses or demotions.

C.2 Seed Stability and Cost-Weight Robustness

Because CAPD is trained, we check whether the main result depends on a single training seed. The main Figure 2 and Table 1 use the pre-specified default seed 3136859. We then retrain CAPD with seeds 3136859, 42, and 2026 while reusing the same deterministic rule-based baselines.

On streamcluster-pressure, all three seeds beat the best classical baseline: the cost reduction ranges from 10.69% to 11.71%, with mean 11.08% and standard deviation 0.45%. This supports streamcluster-pressure as the main positive result. On blackscholes, the default seed improves cost by 0.91%, but the three-seed mean is +0.24% with standard deviation 1.71%; one seed is 2.66% worse than LFU. We therefore interpret blackscholes as a near-tie rather than a robust win. On canneal, all three seeds are worse than LRU, with mean delta +16.81%, confirming it as a stable negative boundary case.

We also reweight the dominant pressure case under alternative cost models. On streamcluster-pressure, CAPD remains better than the best classical baseline under mild, write-heavy, and demotion-heavy cost settings, with reductions from 6.98% to 15.91%. This suggests that the result is not an artifact of a single hand-picked write or demotion weight.